A twenty-nine-variable candidate equation

Positive integer unknowns; exact degree 8; a 135-operation evaluator.

The only unknowns are the twenty-nine separately named positive-integer variables

$$J,F,\alpha,f,g,T,X,Y,\xi,\eta,\zeta,\upsilon,\gamma,\psi,\varphi,\epsilon,$$
$$c,\ell,n,Z,D,V,\delta,$$
$$\kappa,\lambda,\tau,\chi,\theta,Q\in\mathbb{Z}_{>0}.$$

All other symbols below are polynomial abbreviations:

$$b=\beta J,\quad q=1+b,\quad C=q-F-\alpha,\quad U=Y\kappa,$$
$$a=\lambda+Y,\quad s=q\kappa,\quad x=\lambda q,$$
$$W=C(X+Kq)-Fq,\quad m=f^2,$$
$$\sigma=\psi^2,\quad \nu=\upsilon^2,$$
$$E=\lambda,\quad k=\eta+\zeta,$$
$$H=3+4a,\quad \Delta=\delta,\quad L=X+ac,$$
$$M=sb,$$
$$R=[\alpha+(q-F)b](\kappa-1)+J(\mu+\omega q),$$
$$\vartheta=V^2,$$
$$I=\theta,$$
$$\mathcal{P}=Z^2-\sigma(1+\tau),$$
$$\mathcal{D}=D^2-\chi(1+\Delta n),$$
$$\mathcal{Q}=\nu-Q[\nu-\vartheta]-1,$$
$$\mathcal{R}=I^2-Q,$$
$$\mathcal{B}=\psi-M,\quad \mathcal{C}=\varphi-U,\quad \mathcal{G}=\epsilon-R,$$
$$\mathcal{H}=c-\eta-Yk,$$
$$\mathcal{L}=\ell-EYk,$$
$$\mathcal{N}=n-c^2,$$
$$\mathcal{I}=Z-[\psi(\ell+k+\xi-kW)+Yb-Ws],$$
$$\mathcal{J}=D-[\varphi L+H(\varphi\gamma+x+\epsilon+1-k)],$$
$$\mathcal{K}=V-[m(Tc-\epsilon)-c],\quad \mathcal{O}=\delta-H-a^2,$$
$$\mathfrak{J}=\kappa-q^2,\quad \mathcal{F}=\lambda-XY,$$
$$\mathcal{S}=\tau-\ell(\ell+k),$$
$$\mathcal{U}=\chi-\varphi^2,\quad \mathcal{V}=\theta-ng,$$
$$\mathfrak{B}=Q-\Delta(m-1).$$

The single Diophantine equation is

$$\mathcal{P}^2+\mathcal{D}^2+\mathcal{Q}^2+\mathcal{R}^2+\mathcal{B}^2+\mathcal{C}^2+\mathcal{G}^2$$
$$+\mathcal{H}^2+\mathcal{L}^2+\mathcal{N}^2$$
$$+\mathcal{I}^2+\mathcal{J}^2+\mathcal{K}^2+\mathcal{O}^2+\mathfrak{J}^2+\mathcal{F}^2$$
$$+\mathcal{S}^2+\mathcal{U}^2+\mathcal{V}^2+\mathfrak{B}^2=0.$$

The four fixed integers. Every index, set, sum, product and floor below defines a constant, not another unknown. Put

$$\rho=2^{47},\quad B=\rho^{2011},\quad \beta=B-1,$$
$$G_m(x)=\sum_{j=0}^{m-1}x^j,\quad \Lambda=\rho^{1005}.$$

$$\mu=\beta-G_{951}(\rho)$$
$$-\rho^{960}G_{26}(\rho)-\rho^{1004},$$
$$\omega=\beta+8G_{917}(\rho)-4+(2^{45}-4)\Lambda$$
$$+\rho^{960}G_9(\rho)+\rho^{987}G_8(\rho)+\rho^{1004},$$
$$K=\rho^{18}+\rho^{1004}+\rho^{2020}$$
$$+2\Lambda\Phi(1/\rho),$$

where

$$\chi_i=1+2^{10}(1+w_i+2^8n_i$$
$$+2^{17}p_i+2^{25}s_i)\quad(0\le i\le 941),$$
$$\pi(0)=0,\quad \pi(i)=1+\sum_{j=1}^{941}\left\lfloor\frac{2^{44}-1+\chi_j-\chi_i}{2^{44}}\right\rfloor\quad(1\le i\le 941).$$

$$\Phi(x)=\sum_{i=0}^{941}\chi_i x^{\pi(i)}$$
$$+2^{10}x^{969}(x^9+2^{17})G_8(2x)$$
$$+2^{18}x^{942}(x^{18}+2^{17})G_9(2x).$$

The rational intermediate in K cancels exactly; all four results are positive integers.

The 942 indexed fixed tiles. Use the actions u_j,v_j,d_j decoded below. For $d=0,1$, let

$$h(q,b)=2q+b+2,\quad \mathcal{S}_d=\{u_j:d_j=d,\ 0\le j<430\},$$
$$r_{d,q}=205-199d+|\mathcal{S}_d\cap\{0,\dots,q-1\}|.$$

An interval ending at -1 is empty. The table defines t_i for $0\le i\le 941$; all index ranges are fixed:

index i	tile $t_i=(w_i,n_i,p_i,s_i)$
0	$(2,435,3,1)$
$j+1$ $(j=2q+b<430)$	$((1-d_j)r_{0,u_j},h(q,b),d_jr_{1,u_j},v_j)$
$433-2a+b$ $(a,b=0,1)$	$(0,432a+b,0,432a+b)$
$\substack{423+2r_{d,q}+b\\ q\in\mathcal{S}_d,\ d,b=0,1}$	$(dr_{d,q},b,(1-d)r_{d,q},h(q,b))$
$923+2a+b$ $(a,b=0,1)$	$(0,432a+b,0,438)$
927	$(2,434,2,0)$
928	$(3,436,4,4)$
929	$(4,436,4,0)$
$496+t$ $(t=434,435,436)$	$(1,438,1,t)$
$933+3a+b$ $(a,b=0,1,2)$	$(a^2,437,b,437)$

A closed decoder for the 430 actions. Put $D_b(A,k)=\left\lfloor A/b^k\right\rfloor \bmod b$, where $a \bmod b = a-b\lfloor a/b\rfloor$. For $0\le q<215$, $b=0,1$, $j=2q+b$:

$$\mathcal{N}=\mathcal{A}\bmod 2^{636},\quad \mathcal{T}=\left\lfloor\mathcal{A}/2^{636}\right\rfloor,$$
$$\nu_j=D_2(\mathcal{N},j),\quad \kappa_j=(\sigma_j+2)^{1-\nu_j}\quad(0\le j<430),$$
$$\sigma_j=\sum_{k=0}^{j-1}\nu_k,\quad \Pi_j=\prod_{k=0}^{j-1}\kappa_k\quad(0\le j\le 430),$$
$$m_q=D_2(\mathcal{N},430+q),\quad \tau_q=\sum_{k=0}^{q-1}m_k,$$
$$F_q=14+m_q(D_{16}(\lfloor\mathcal{T}/\Pi_{430}\rfloor,\tau_q)-14),$$
$$u_j=(\sigma_{j+1}-(\lfloor\mathcal{T}/\Pi_j\rfloor \bmod \kappa_j)) \bmod 216,$$
$$v_j=D_2(F_q,2b),\quad d_j=D_2(F_q,2b+1).$$

Empty sums/products are 0/1. Here $(v_0,d_0,u_0)=(1,1,1)$; the constructor tile uses this actual first target, including its state renaming.

The integer $\mathcal{A}$ in base 10. Its complete decimal expansion is the concatenation of these lines:

```
12889260410359561746736540743515845134244260640012702033212621
95398074180784058571518488282119861023093355389406939414042892
97455773992193784795705001618777025966369137954859764532832722
75411843410809038713502633204617333246733616940964578272360901
56186216508504953649192752154519268024509952082950238865468307
22999632482769253980274091217460707829023970817244790340704449
09085581271192071656170732004937337082171141082486384269072884
72778087419663417942892520175630649792329265834490253035577533
96449689808375788457488971017043689875433162972390177936123032
89025657757489526342497438772045714099661532481123050235898973
27443609903800541199351405985754278307227777584454508422717998
33411356440933534607489268032843 7
```

The fixed integer $\mathcal{A}$ above is written in base 10. This equation is a candidate: its model witness transport has been checked, but its native intuitionistic PRA equivalence has not passed the guarded replay. Its exact degree is certified by integer-polynomial calculations; the 135-operation count is for the unshifted circuit. Neither count is a minimality claim.

Exact 135-operation circuit

Each numbered line defines one gate over the integers. The inputs are the twenty-nine unknowns and the four fixed integers above. The ASCII names `alpha, xi, eta, zeta, upsilon, gamma, vartheta, psi, varphi, epsilon, beta, mu, omega, ell, Delta, delta, kappa, lambda, sigma, tau, chi, theta, nu` refer to their displayed counterparts; `g` is the unknown g. The last gate is zero exactly when the displayed equation holds. The circuit inputs `cNamed, ellNamed` are c, ℓ; gate `a` is the displayed abbreviation; gate `WComputed` is the displayed abbreviation $W = C(X + Kq) - Fq$, and gate `m` is $m = f^2$. The inputs `n, Z, D, V, delta` are the displayed n, Z, D, V, δ; `kappa, lambda, tau, chi, theta, Q` are $\kappa, \lambda, \tau, \chi, \theta, Q$. Gates `q3, g55, g39, y2, g80` compute $s, x, \sigma, \nu, \vartheta$. Gates `E, I` compute XY, ng; the display uses $E = \lambda$, $I = \theta$. Intermediate gate names below are local abbreviations, not additional unknowns.

```
1.  b = J * beta
2.  q = 1 + b
3.  q2 = q * q
4.  q3 = q * kappa
5.  U = Y * kappa
6.  qF = q - F
7.  C = qF - alpha
8.  E = X * Y
9.  k = eta + zeta
10.  kY = Y * k
11.  c = eta + kY
12.  c2 = cNamed * cNamed
13.  a = lambda + Y
14.  g15 = 4 * a
15.  H = 3 + g15
16.  g17 = a * a
17.  DeltaComputed = H + g17
18.  g19 = a * cNamed
19.  L = X + g19
20.  ell = lambda * kY
21.  M = q3 * b
22.  g25 = K * q
23.  g26 = X + g25
24.  g27 = C * g26
25.  g28 = F * q
26.  WComputed = g27 - g28
27.  g30 = ellNamed + k
28.  g31 = g30 + xi
29.  root_skew = k * WComputed
30.  root_inner = g31 - root_skew
31.  root_scaled = psi * root_inner
32.  g33 = Y * b
33.  root_btq3 = WComputed * q3
34.  root_sum = root_scaled + g33
35.  ZComputed = root_sum - root_btq3
36.  g38 = Z * Z
37.  g39 = psi * psi
38.  g40 = ellNamed * g30
39.  g41 = 1 + tau
40.  g42 = g39 * g41
41.  A0 = g38 - g42
42.  g44 = qF * b
43.  rankseed = alpha + g44
44.  g46 = kappa - 1
45.  g47 = rankseed * g46
46.  g48 = omega * q
47.  g49 = mu + g48
48.  g50 = J * g49
49.  R = g47 + g50
50.  g52 = k - epsilon
51.  r0 = g52 - 1
52.  g54 = varphi * gamma
53.  g55 = lambda * q
54.  g56 = g54 + g55
55.  shift = g56 - r0
56.  g58 = varphi * L
57.  g59 = H * shift
58.  DstarComputed = g58 + g59
59.  g61 = D * D
60.  g62 = varphi * varphi
61.  g63 = delta * n
62.  g64 = 1 + g63
63.  g65 = chi * g64
64.  A1 = g61 - g65
65.  m = f * f
66.  rho = m - 1
67.  QComputed = delta * rho
68.  I = n * g
69.  g71 = theta * theta
70.  g72 = g71 - Q
71.  g74 = T * cNamed
72.  g75 = g74 - epsilon
73.  g76 = m * g75
74.  VComputed = g76 - cNamed
75.  y2 = upsilon * upsilon
76.  g80 = V * V
77.  g81 = y2 - g80
78.  g82 = Q * g81
79.  Fa = y2 - g82
80.  auxResidual = Fa - 1
81.  firstSquare = A0 * A0
82.  mainSquare = A1 * A1
83.  auxSquare = auxResidual * auxResidual
84.  strongSquare = g72 * g72
85.  firstMain = firstSquare + mainSquare
86.  firstMainAux = firstMain + auxSquare
87.  sumSquares = firstMainAux + strongSquare
88.  scaleDiff = psi - M
89.  scaleCheck = scaleDiff * scaleDiff
90.  final = sumSquares + scaleCheck
91.  mainDiff = varphi - U
92.  mainCheck = mainDiff * mainDiff
93.  result16 = final + mainCheck
94.  rankDiff = epsilon - R
95.  rankCheck = rankDiff * rankDiff
96.  result17 = result16 + rankCheck
97.  cDifference = cNamed - c
98.  cConstraint = cDifference * cDifference
99.  ellDifference = ellNamed - ell
100.  ellConstraint = ellDifference *
ellDifference
101.  addC = result17 + cConstraint
102.  addEll = addC + ellConstraint
103.  cSquareDifference = n - c2
104.  cSquareConstraint = cSquareDifference *
cSquareDifference
105.  addCSquare = addEll + cSquareConstraint
106.  zDifference = Z - ZComputed
107.  zConstraint = zDifference * zDifference
108.  dDifference = D - DstarComputed
109.  dConstraint = dDifference * dDifference
110.  vDifference = V - VComputed
111.  vConstraint = vDifference * vDifference
112.  deltaDifference = delta - DeltaComputed
113.  deltaConstraint = deltaDifference *
deltaDifference
114.  addZConstraint = addCSquare + zConstraint
115.  addDConstraint = addZConstraint +
dConstraint
116.  addVConstraint = addDConstraint +
vConstraint
117.  addDELTAConstraint = addVConstraint +
deltaConstraint
118.  qSquareDifference = kappa - q2
119.  qSquareConstraint = qSquareDifference *
qSquareDifference
120.  product2Difference = lambda - E
121.  product2Constraint = product2Difference *
product2Difference
122.  ellProductDifference = tau - g40
123.  ellProductConstraint =
ellProductDifference * ellProductDifference
124.  varphiSquareDifference = chi - g62
125.  varphiSquareConstraint =
varphiSquareDifference * varphiSquareDifference
126.  scaledIndexDifference = theta - I
127.  scaledIndexConstraint =
scaledIndexDifference * scaledIndexDifference
128.  qSignedDifference = Q - QComputed
129.  qSignedConstraint = qSignedDifference *
qSignedDifference
130.  add_qSquare = addDELTAConstraint +
qSquareConstraint
131.  add_product2 = add_qSquare +
product2Constraint
132.  add_ellProduct = add_product2 +
ellProductConstraint
133.  add_varphiSquare = add_ellProduct +
varphiSquareConstraint
134.  add_scaledIndex = add_varphiSquare +
scaledIndexConstraint
135.  add_qSigned = add_scaledIndex +
qSignedConstraint
```